

Laporan Simulasi Agentic AI Control Soccer Pioneer P3DX

Author: Yoel Yohendy
NRP : 5022211133
Teacher: Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Class name: Sistem Robot Otonom
YouTube video link: <https://youtu.be/fx3pPcAbVuA>
GitHub link: <https://github.com/miauah/COPSIM-AGENTIC-SOCCER-SIMULATION.git>

Laporan ini membahas implementasi sistem Agentic AI pada robot Pioneer P3DX menggunakan CoppeliaSim dan integrasi model Large Language Model (LLM) Qwen secara lokal. Sistem ini dikembangkan agar robot dapat menerima dan menjalankan instruksi bahasa natural dengan memecahnya menjadi serangkaian action berdasarkan pembacaan state.

Bab 1: Pendahuluan

Berikut adalah ketentuan dari Simulasi:

- **Harus menggunakan CoppeliaSim** sebagai lingkungan simulasi.
- **Python 1 file berargumen (*state*, *action*)**, yang berfungsi sebagai antarmuka (interface) komunikasi antara CoppeliaSim dan AI.
- **Feedback system (*close loop*)**, di mana aksi robot dievaluasi kembali oleh pembacaan sensor secara terus-menerus.
- **Robot Mobile**, dalam hal ini digunakan robot beroda Pioneer P3DX.
- **Menjalankan misi tertentu**, yaitu misi bermain bola dan mencetak gol.
- **Menggunakan AI LLM** sebagai otak pengambil keputusan (*inference system*).

Sistem ini memodelkan dua variabel utama:

1. **State (Input Sensor):** Meliputi koordinat posisi robot, posisi gawang, kecepatan robot, serta pembacaan orientasi.
2. **Action (Output Aksi):** Meliputi pergerakan di *joint space* dan aksi *preset open-loop* seperti perintah maju dalam meter, rotasi dalam derajat, atau mekanisme mekanik untuk menendang.

Bab 2: Metodologi Pengerjaan

2.1 AI dan Integrasi LM Studio

Sistem ini menggunakan LLM Qwen 3.5 4B yang dijalankan secara lokal menggunakan aplikasi LM Studio. LM Studio dikonfigurasi untuk menjalankan server lokal yang menyediakan API yang dapat dipanggil oleh program. Hal ini memungkinkan program Python untuk mengirimkan *prompt* dan menerima respons *JSON* sebagai perantara ke simulasi.

2.2 Workflow Simulasi

Alur kerja sistem ini berjalan dalam siklus observasi dan tindakan sehingga bekerja secara close loop :

1. Pengguna memberikan instruksi bahasa natural melalui suatu GUI (contoh: "make a goal").
2. Program harness meminta data State terbaru dari CoppeliaSim.
3. Data *State* ini dibungkus beserta instruksi dan dikirim sebagai *prompt* ke LLM Qwen.

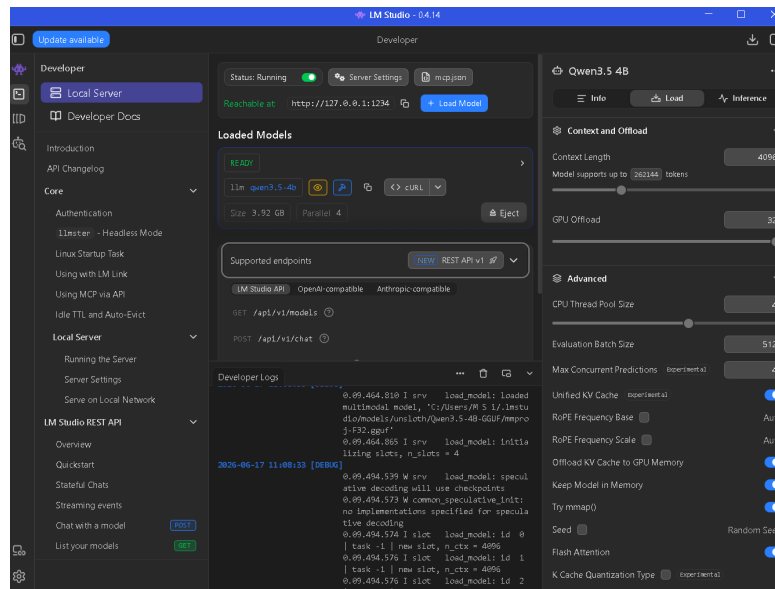


Figure 1: Tampilan server LM Studio dan log respons dari LLM Qwen.

- LLM menganalisis data, memberikan pemikiran logis (*thinking reasoning*), dan mengembalikan struktur data JSON berisi *Action* yang harus diambil (contoh: `{"command": "rotate", "angle": -1.2}`).
- Program mengeksekusi *Action* tersebut pada CoppeliaSim melalui *subprocess*, kemudian loop kembali ke langkah 2.

2.3 Desain Map dan Objek Simulasi

Lingkungan simulasi dirancang berupa lapangan sepak bola hijau sederhana yang memiliki tiga objek kunci:

- PioneerP3DX:** Robot beroda yang dilengkapi dengan pendorong linier (*prismatic joint*) di bagian depannya untuk mekanisme menendang.
- Sphere:** Sebagai bola sepak
- Goalpost:** Gawang tempat bola harus dimasukkan. Letak gawang diset statis pada koordinat spasial $X \in [2.0, 2.5]$ dan $Y \in [-1.0, 1.0]$.

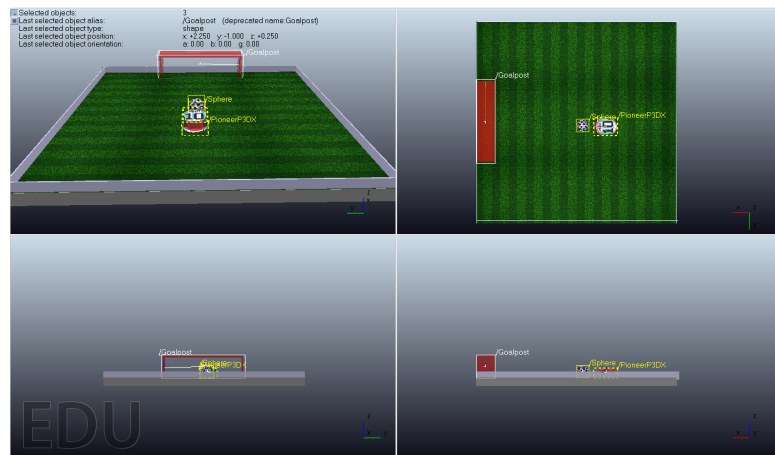


Figure 2: Desain lapangan, letak gawang, dan posisi awal Pioneer P3DX di CoppeliaSim.

2.4 Struktur Program (*llm_soccer_agent.py* & *soccer_harness.py*)

Program terbagi menjadi dua komponen utama agar modul antarmuka dan kontrol terpisah dengan baik:

- `llm_soccer_agent.py`: Berfungsi sebagai *backend* yang langsung terhubung ke CoppeliaSim melalui ZeroMQ (ZMQ). Skrip ini memproses calling subfungsi secara *open-loop* untuk perintah presisi seperti `move_forward`, `rotate`, `move_backward`, `celebrate`, `kick`, dan `get_data`.
- `soccer_harness.py`: Bertindak sebagai interface (UI) jembatan antara LLM dan user. Skrip ini mengelola UI pengguna untuk input prompt, mem-parsing format *JSON* dari respons LLM, dan menjalankan program *agent* sebagai subprocess.

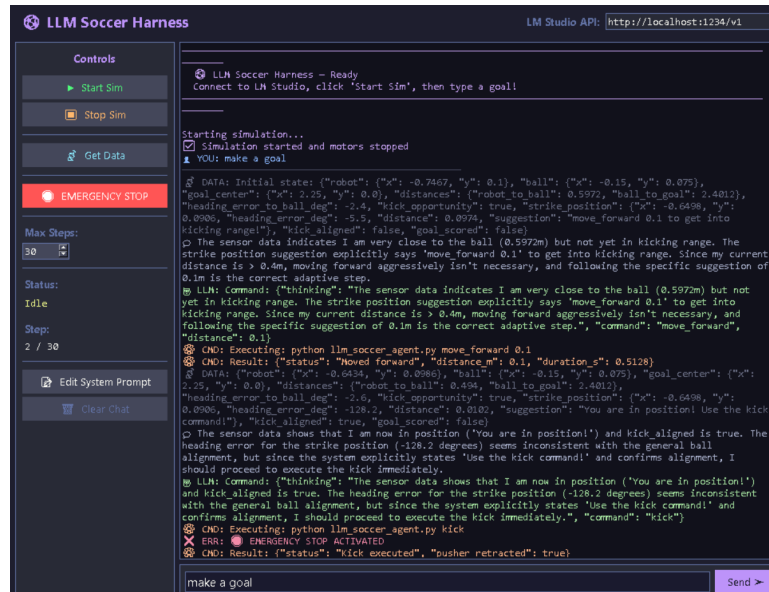


Figure 3: Bentuk aplikasi Grafik Interface AI dan contoh penggunaan.

Bab 3: Hasil Simulasi dan Kesimpulan

Secara keseluruhan, simulasi membuktikan bahwa pendekatan *Agentic AI* sangat kapabel untuk mengontrol robot. Sistem mampu memahami dan menerjemahkan instruksi pengguna seperti *"score a goal"* atau *"go to the goalpost"* ke dalam rentetan eksekusi mekanik secara mandiri. Namun, selama proses pengujian, ditemukan beberapa kelemahan dan karakteristik sistem yang telah dievaluasi:

1. **Tendensi *Overshoot* dan Tabrakan Fisik:** Karena fisika CoppeliaSim menyimulasikan inersia dan dimensi fisik robot, perintah maju dengan *step* yang terlalu besar misal: maju 0.5 meter saat sudah sangat dekat akan menyebabkan *overshoot* sehingga robot tidak sengaja menyenggol dan menjauhkan bola. Hal ini diselesaikan dengan mengatur `STRIKE.OFFSET` yaitu batas aman jarak ke bola secara akurat di angka 0.50m. Posisi ini memberikan celah antara body robot dan bola, namun tetap membiarkan mekanisme pendorong (kicker) menjangkau pusat bola untuk tendangan yang kuat.
2. **Kecenderungan *Micro-adjustment*:** Kelemahan umum dari instruksi LLM tanpa batasan adalah interpretasi matematis yang sangat literal. Ketika terdapat deviasi orientasi kecil seperti 0.4 derajat, LLM mencoba memutar robot untuk mencapai posisi absolut 0.0 derajat. Hal ini menjebak AI dalam pengulangan koreksi yang membuang banyak waktu. Isu ini diatasi dengan merancang sistem toleransi pada sistem saran, di mana deviasi di bawah 5 derajat dianggap sejajar sempurna.
3. **Kegagalan *Parsing Output* (Halusinasi Format LLM):** Penggunaan model LLM lokal berukuran kecil seperti Qwen 4B terkadang memicu kegagalan sistem dalam mengikuti format respons JSON yang ketat. Model sering kali menambahkan teks ekstra yang menyebabkan program gagal mem-*parsing* perintah, meskipun logika pergerakannya benar.

Kesimpulan & Saran:

Simulasi ini secara sukses membuktikan kelayakan LLM sebagai decision maker pada robot. Walaupun begitu, disarankan untuk tidak sepenuhnya menyerahkan kontrol level-rendah kepada LLM karena masalah latensi dan inefisiensi. Sebagai saran untuk pengembangan selanjutnya, LLM sebaiknya difokuskan untuk menghasilkan keputusan tingkat tinggi, dan menyerahkan navigasi mikro serta stabilisasi gerak kepada algoritma kontrol closed loop deterministik.